

Inter thread communication & Locks

Inter-thread communication is required when one thread cannot continue safely until another thread changes some shared state.

Instead of repeatedly checking the shared state, a thread can pause and allow another thread to perform the required work. Java provides three methods for this purpose:

```
wait();  
notify();  
notifyAll();
```

These methods work with an object's intrinsic monitor lock.

1. The Producer–Consumer Problem

Consider a shared box that can hold only one item:

```
class Box {  
    private int item;  
    private boolean hasItem = false;  
}
```

Two threads use this box:

- The **producer** places an item into the box.
- The **consumer** removes the item from the box.

The rules are:

```
If the box is empty, the consumer must wait.  
If the box is full, the producer must wait.
```

The producer and consumer are not only competing for access to the shared object. They also depend on the state created by each other.

2. Why `synchronized` Alone Is Not Enough

The `synchronized` keyword ensures that only one thread can execute a synchronized section for the same monitor at a time.

```
class Box {
    private int item;
    private boolean hasItem = false;

    synchronized void produce(int value) {
        item = value;
        hasItem = true;
        System.out.println("Produced: " + item);
    }

    synchronized void consume() {
        System.out.println("Consumed: " + item);
        hasItem = false;
    }
}
```

This code protects the shared data from simultaneous access, but it does not enforce the producer-consumer rules.

Possible problems include:

- The producer may overwrite an item before it is consumed.
- The consumer may consume when the box is empty.
- The consumer may run repeatedly without waiting for a new item.

Therefore:

`synchronized` provides mutual exclusion and memory visibility, but it does not automatically provide condition-based waiting.

3. Busy Waiting

One possible approach is to keep checking the condition continuously:

```
while (!hasItem) {  
    // keep checking  
}
```

This technique is called **busy waiting** or **spinning**.

The thread repeatedly checks:

```
Is the item available?  
Is the item available?  
Is the item available?
```

The thread is not performing useful work, but it continues consuming CPU time.

A producer–consumer implementation based on busy waiting may look like this:

```
class Box {  
    private int item;  
    private boolean hasItem = false;  
  
    synchronized void produce(int value) {  
        while (hasItem) {  
            // keep looping  
        }  
  
        item = value;  
        hasItem = true;  
        System.out.println("Produced: " + item);  
    }  
  
    synchronized void consume() {  
        while (!hasItem) {
```

```
// keep looping
}

System.out.println("Consumed: " + item);
hasItem = false;
}
}
```

This version has an additional serious problem: the thread holds the monitor lock while looping. The other thread cannot enter the synchronized method to change the condition.

Even declaring the shared flag as `volatile` does not completely solve the problem.

`volatile` can make the latest value visible between threads, but it does not:

- Pause a thread
- Release a monitor lock
- Provide notification
- Make multiple operations atomic

4. The Better Approach

Instead of repeatedly checking the condition, a thread should:

1. Check whether it can proceed.
2. Pause if the condition is not satisfied.
3. Release the monitor so another thread can change the condition.
4. Wake up when another thread signals that the state may have changed.
5. Recheck the condition before continuing.

Java supports this workflow through:

```
wait();
notify();
```

```
notifyAll();
```

5. Mental Model

Most inter-thread communication problems contain three elements:

1. Shared resource
2. Condition
3. Waiting and notification

For the producer–consumer problem:

Shared resource: Box
Condition: Whether the box contains an item
Waiting: Consumer waits when the box is empty
Notification: Producer notifies after adding an item

The reverse also applies:

Waiting: Producer waits when the box is full
Notification: Consumer notifies after removing the item

```
wait()
```

6. What Does `wait()` Do?

When a thread calls `wait()` on an object:

1. It releases that object's monitor lock.
2. It enters the waiting set associated with that object.
3. It enters the `WAITING` state for an untimed `wait()`.
4. It remains there until it is notified, interrupted, or wakes spuriously.
5. Before returning from `wait()`, it must acquire the same monitor lock again.

Example:

```
synchronized (lock) {
    while (!condition) {
        lock.wait();
    }

    // Continue after the condition becomes true
}
```

Important

`wait()` releases only the monitor on which it is called.

If the thread holds locks for other objects, those locks are not automatically released.

7. Why Releasing the Lock Is Essential

Suppose the consumer acquires the monitor and discovers that the box is empty:

```
synchronized (lock) {
    while (!hasItem) {
        lock.wait();
    }
}
```

If `wait()` did not release the monitor:

```
Consumer holds the lock
→ Producer cannot enter
→ Producer cannot add an item
→ Condition can never change
```

The system would become stuck.

Therefore, `wait()` performs two connected actions:

Release the monitor
+
Pause the current thread

This allows another thread to acquire the same monitor and change the shared condition.

8. Monitor Ownership Rule

A thread can call `wait()` only when it owns the monitor of the same object.

Correct:

```
synchronized (lock) {  
    lock.wait();  
}
```

Incorrect:

```
lock.wait();
```

The incorrect version throws:

```
IllegalMonitorStateException
```

A thread cannot wait on or release a monitor that it does not own.

Inside a Synchronized Instance Method

A synchronized instance method locks the current object, represented by `this`.

```
synchronized void consume() throws InterruptedException {  
    wait();  
}
```

Here, `wait()` is equivalent to:

```
this.wait();
```

The thread already owns the monitor of `this`, so the call is valid.

9. Monitor Lock and Waiting Set

A useful conceptual model is that every Java object can participate in monitor-based synchronization through:

1. An intrinsic monitor lock
2. A waiting set associated with that monitor

When a thread calls:

```
lock.wait();
```

the following happens:

```
Thread releases lock's monitor
→ enters lock's waiting set
→ waits for a signal or another permitted wake-up
```

The waiting set belongs to the object, not to the thread.

10. Simple `wait()` Example

```
class Box {
    private int item;
    private boolean hasItem = false;

    synchronized void consume() throws InterruptedException {
        while (!hasItem) {
            wait();
        }
    }
}
```



```
        System.out.println("Consumed: " + item);
        hasItem = false;
    }
}
```

The consumer releases the `Box` monitor while waiting. This allows the producer to enter another synchronized method on the same object and produce an item.

`notify()`

11. What Does `notify()` Do?

When a thread calls:

```
lock.notify();
```

one thread waiting on that object's monitor is selected for wake-up.

The selected thread does not continue immediately.

It must first reacquire the same monitor.

Conceptually:

```
One waiting thread is selected
→ it becomes eligible to compete for the monitor
→ notifying thread continues holding the monitor
→ notifying thread exits the synchronized section
→ awakened thread may acquire the monitor
→ awakened thread returns from wait()
```

12. `notify()` Does Not Transfer the Lock

A common misunderstanding is that `notify()` immediately hands the lock to the waiting thread.

It does not.

```
synchronized (lock) {
    lock.notify();

    // Current thread still owns the monitor here
}
```

The awakened thread cannot continue past `wait()` until the notifying thread releases the monitor by leaving the synchronized section.

13. Which Thread Does `notify()` Select?

If several threads are waiting on the same monitor, `notify()` selects one of them.

The selection is arbitrary and should be treated as non-deterministic.

The programmer cannot safely assume that a particular producer or consumer will be selected.

14. Monitor Ownership for `notify()`

Just like `wait()`, `notify()` must be called while owning the monitor of the same object.

Correct:

```
synchronized (lock) {
    lock.notify();
}
```

Incorrect:

```
lock.notify();
```

The incorrect version throws:

```
IllegalMonitorStateException
```

The same rule applies to `notifyAll()`.

`notifyAll()`

15. What Does `notifyAll()` Do?

`notifyAll()` wakes all threads waiting on the same object's monitor.

```
lock.notifyAll();
```

This does not mean that all threads execute simultaneously.

The process is:

```
All waiting threads become eligible to compete for the monitor
→ only one thread acquires the monitor at a time
→ each awakened thread rechecks its condition
→ threads whose conditions are still false wait again
```

16. Why `notify()` Can Be Risky

Suppose producers and consumers are both waiting on the same `Box` monitor:

```
Producer 1 waits for space
Producer 2 waits for space
Consumer 1 waits for an item
Consumer 2 waits for an item
```

If the box is full and a consumer removes the item, a producer should wake up.

However, `notify()` may select another consumer.

That consumer checks the condition, discovers that the box is empty, and waits again. Meanwhile, all producers may remain asleep even though space is available.

This can leave the system stuck even though progress is logically possible.

`notifyAll()` avoids relying on the runtime to select the correct category of thread. Every waiting thread gets an opportunity to recheck its own condition.

17. `notify()` vs `notifyAll()`

Method	Behaviour	Advantage	Risk or Cost
<code>notify()</code>	Wakes one arbitrary waiting thread	Lower wake-up overhead	May wake a thread whose condition is still false
<code>notifyAll()</code>	Wakes all waiting threads	Safer when different conditions use the same monitor	More threads wake and compete unnecessarily

Use `notify()` when:

- Only one category of thread is waiting.
- Any selected waiting thread can safely proceed.
- The waiting conditions are simple and well controlled.

Use `notifyAll()` when:

- Different types of threads wait on the same monitor.
- Multiple conditions are involved.
- Correct thread selection cannot be guaranteed.
- Safety and clarity are more important than minor wake-up overhead.

For basic monitor-based application code, `notifyAll()` is usually the safer default.

Complete Producer–Consumer Implementation

18. Safe `Box` Class

```
class Box {
    private int item;
    private boolean hasItem = false;

    public synchronized void produce(int value)
        throws InterruptedException {

        while (hasItem) {
            wait();
        }

        item = value;
        hasItem = true;

        System.out.println(
            Thread.currentThread().getName()
                + " produced: " + item
        );

        notifyAll();
    }

    public synchronized int consume()
        throws InterruptedException {

        while (!hasItem) {
            wait();
        }

        int value = item;
        hasItem = false;

        System.out.println(
            Thread.currentThread().getName()
                + " consumed: " + value
        );
    }
}
```

```
);

    notifyAll();
    return value;
}
}
```

Producer Condition

```
while (hasItem) {
    wait();
}
```

The producer waits while the box is full.

Consumer Condition

```
while (!hasItem) {
    wait();
}
```

The consumer waits while the box is empty.

Notification

After changing the shared state, each method calls:

```
notifyAll();
```

The shared state must be changed before notification so that awakened threads can observe the new condition after reacquiring the monitor.

19. Runnable Example

```
public class Main {

    public static void main(String[] args) {
        Box box = new Box();

        Thread producer = new Thread(() -> {
            for (int value = 1; value <= 5; value++) {
                try {
                    box.produce(value);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }, "Producer");

        Thread consumer = new Thread(() -> {
            for (int count = 1; count <= 5; count++) {
                try {
                    box.consume();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }, "Consumer");

        producer.start();
        consumer.start();
    }
}
```

A possible output is:

```
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
```

The exact scheduling order is controlled by the JVM and operating system, but the rules of the box remain protected.

Why These Methods Belong to **Object**

20. The Lock Belongs to an Object

The following methods are declared in the **Object** class:

```
wait();
notify();
notifyAll();
```

They are not declared in **Thread** because the waiting mechanism is associated with an object's monitor.

In Java, a thread does not wait on another thread. It waits for a condition protected by a particular object's monitor.

```
synchronized (box) {
    box.wait();
}
```

Here:

- **box** is the monitor object.
- The current thread owns **box**'s monitor.
- The current thread enters **box**'s waiting set.
- Another thread must synchronize on the same **box** object.

- That thread must call `box.notify()` or `box.notifyAll()`.

Calling `wait()` on one object and `notify()` on another object does not establish communication between their waiting sets.

sleep() vs wait()

21. Key Differences

Feature	<code>Thread.sleep()</code>	<code>wait()</code>
Declared in	<code>Thread</code>	<code>Object</code>
Main purpose	Pause for a duration	Wait for a condition to change
Releases monitor lock	No	Yes
Requires monitor ownership	No	Yes
Common thread state	<code>TIMED_WAITING</code>	<code>WAITING</code>
Wakes because of	Time completion or interruption	Notification, interruption, timeout, or spurious wakeup

sleep()

```
synchronized (lock) {
    Thread.sleep(1000);
}
```

The thread pauses, but it continues holding `lock` during the sleep period.

wait()

```
synchronized (lock) {
    lock.wait();
}
```

The thread pauses and releases `lock`, allowing another thread to enter a synchronized section guarded by the same monitor.

Spurious Wakeups

22. Why `if` Is Incorrect

A thread waiting through `wait()` is permitted to wake up even when no useful condition change has occurred.

This is known as a **spurious wakeup**.

A thread may also receive a notification even though the condition it needs is still false.

Therefore, waking up does not prove that the required condition is true.

Incorrect:

```
if (!hasItem) {  
    wait();  
}  
  
System.out.println("Consumed: " + item);
```

If the thread wakes while `hasItem` is still `false`, it continues with invalid state.

Correct:

```
while (!hasItem) {  
    wait();  
}  
  
System.out.println("Consumed: " + item);
```

After every wake-up, the condition is checked again.

23. The Golden Rule

Always place `wait()` inside a loop that checks the guarded condition:

```
while (!condition) {  
    wait();  
}
```

Do not use:

```
if (!condition) {  
    wait();  
}
```

The loop handles:

- Spurious wakeups
- A notification received by the wrong type of waiting thread
- The condition changing again before the awakened thread reacquires the lock
- Multiple threads competing for the same shared state

Guarded Blocks

24. Guarded Block Pattern

A **guarded block** executes only when a required condition is true. Otherwise, the thread waits.

```
synchronized (lock) {  
    while (!condition) {  
        lock.wait();  
    }  
  
    // Perform the protected operation  
  
    lock.notifyAll();  
}
```

The condition is called the **guard condition**.

For the consumer:

```
while (!hasItem) {
    wait();
}
```

For the producer:

```
while (hasItem) {
    wait();
}
```

A guarded block combines:

```
Monitor ownership
+ condition checking
+ waiting
+ state modification
+ notification
```

Handling Interruption

25. **InterruptedException**

`wait()` declares `InterruptedException` because another thread may interrupt a waiting thread.

```
try {
    box.consume();
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

When the method cannot completely handle the interruption, a common practice is to restore the interrupt status:

```
Thread.currentThread().interrupt();
```

The thread can then stop its current task or allow a higher-level component to decide how to respond.

Complete Execution Flow

26. Consumer Waits and Producer Notifies

1. Consumer acquires the Box monitor.
2. Consumer checks `hasItem`.
3. `hasItem` is false.
4. Consumer calls `wait()`.
5. Consumer releases the Box monitor.
6. Consumer enters the Box waiting set.
7. Producer acquires the Box monitor.
8. Producer adds an item.
9. Producer sets `hasItem` to true.
10. Producer calls `notifyAll()`.
11. Consumer becomes eligible to compete for the monitor.
12. Producer exits the synchronized method.
13. Producer releases the monitor.
14. Consumer reacquires the monitor.
15. Consumer returns from `wait()`.
16. Consumer rechecks `hasItem`.
17. Consumer removes the item.

The awakened thread continues from the `wait()` call only after reacquiring the monitor.

Important Rules

27. Rules to Remember

1. Call `wait()`, `notify()`, and `notifyAll()` only while owning the same object's monitor.
2. Call these methods on the same object used for synchronization.
3. `wait()` releases the relevant monitor; `sleep()` does not.
4. `notify()` and `notifyAll()` do not immediately release the monitor.
5. A notified thread must reacquire the monitor before continuing.
6. Always check the condition using `while`, not `if`.
7. Change the shared state before sending the notification.
8. Prefer `notifyAll()` when multiple conditions or different waiting-thread types share one monitor.
9. Keep the condition check and state modification inside the same synchronized region.
10. Handle `InterruptedException` deliberately instead of silently ignoring it.

Final Summary

Inter-thread communication is needed when threads depend on changes made by one another.

```
synchronized
→ protects the shared state

wait()
→ releases the monitor and pauses the thread

notify()
→ wakes one arbitrary waiting thread

notifyAll()
→ wakes all waiting threads

while condition check
→ ensures the thread proceeds only when the state is valid
```

The standard guarded-block pattern is:

```
synchronized (lock) {  
    while (!condition) {  
        lock.wait();  
    }  
  
    // Use or modify shared state  
  
    lock.notifyAll();  
}
```

This pattern prevents busy waiting, maintains thread safety, and allows cooperating threads to coordinate correctly around shared state.